FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE
3

Data Mining

Group Name:	Michael Dominic L. Liggayu
Members:	**Name** / **Role**
Section:	An42
Professor:	Ms. Crisola G. Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of the following algorithms: Decision Tree, K-Mean, Apriori.
- Implement Decision Tree Algorithm by designing and developing a simple predictor program using Python.

IV. BACKGROUND INFORMATION

In order to accomplish this exercise, the student must have a clear understanding of the following topics:

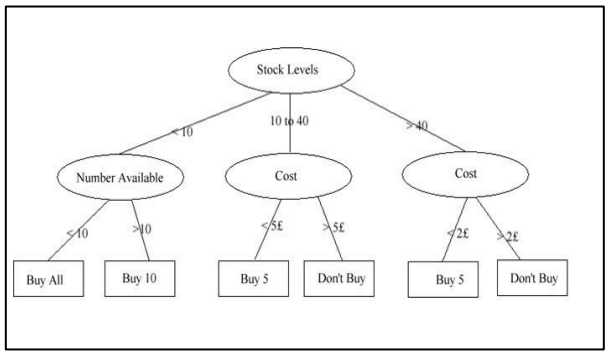
Decision Tree Algorithm

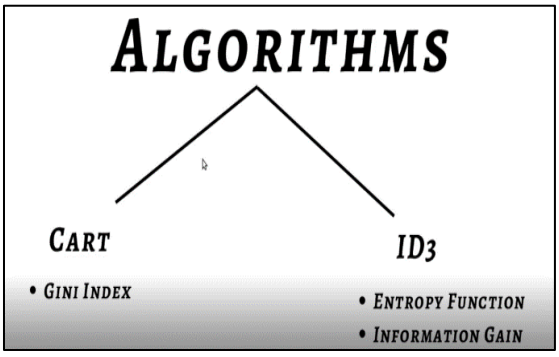
- Related to most of the other techniques (primarily classification and prediction), the decision tree can be used either as a part of the selection criteria, or to support the use and selection of specific data within the overall structure.

A Decision Tree is a tree:

- where each node represents a feature (attribute)
- each branch represents a decision (rule)
- each leaf represents an outcome

Example of Decision Tree





https://octaviansima.wordpress.com/2011/03/25/decision-trees-c4-5/

What is an ID3 Algorithm?

- Stands for Iterative Dichotomiser 3
- An algorithm developed by J.Ross Quinlan
- The core algorithm for building decision trees.
- Supervised learning algorithm used for classification problems.
- It is a classification algorithm that follows a greedy approach by selecting a best attribute that yields maximum Information Gain(IG) or minimum Entropy(H).

STEPS in Creating the Decision Tree
Important Concepts:

- Calculate **Entropy (the amount of uncertainty in dataset : the number of positive and negative evidences).**

$$Entropy = \frac{-p}{p+n} log_2(\frac{p}{p+n}) - \frac{n}{p+n} log_2(\frac{n}{p+n})$$

** Entropy(S) – the entropy of the entire data set.
** Computing the entropy of each attribute

- Calculate **Average Information**.

$$I(Attribute) = \sum \frac{p_i + n_i}{p+n} Entropy(A)$$

- Calculate **Information Gain** (Difference in Entropy before and after splitting dataset on attribute A)

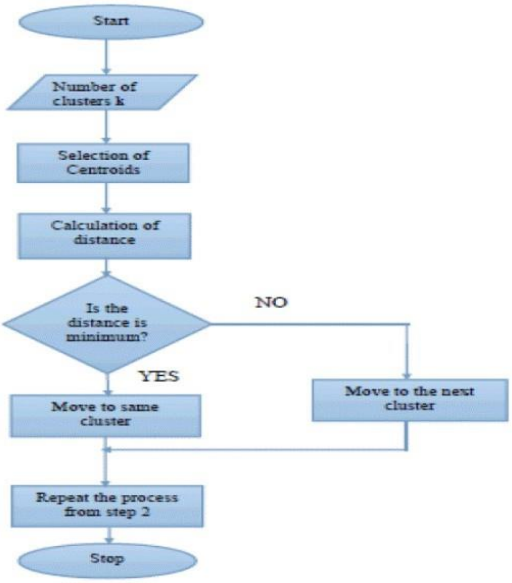
$$Gain = Entropy(S) - I(Attribute)$$

K-Means Algorithm

K-means creates k groups from a set of objects so that the members of a group are more similar. It's a popular cluster analysis technique for exploring a dataset.
Cluster analysis is a family of algorithms designed to form groups such that the group members are more similar versus non-group members. Clusters and groups are synonymous in the world of cluster analysis.

K-means Clustering Algorithm

1. Choose a value of k number of clusters to be formed.
2. Randomly select k data points from the data set as the initial cluster centroids/centers
3. For each datapoint:
   Compute the distance between thedatapoint and the cluster centroid
   Assign the datapoint to the closest centroid
4. 4. For each cluster calculate the new mean based on the datapoints in the cluster.
5. 5. Repeat 3 & 4 steps until mean of the clusters stops changing or maximum number of iterations reached.



https://miro.medium.com/max/1022/1*ZN6cDw34wEO-dxydDwY4vA.png

Apriori Algorithm

What does it do?
It learns association rules and is applied to a database containing a large number of transactions.

What is Association Rule?
It is a data mining technique for learning correlations and relations among variables in a database.

Apriori – Example
In a database full of supermarket transactions, think of a database as a giant spreadsheet where each row is a customer transaction and every column represents a different grocery item.

Transaction ID	Chips	Dip	Soda	Apples	Milk
1	X	X	X		
2	X	X			X
3	X		X		

By applying the Apriori algorithm, we can learn the grocery items that are purchased together a.k.a association rule. One can find those items that tend to be purchased together more frequently than other items — the ultimate goal being to get shoppers to buy more. Together, these items are called “itemsets”.

How Apriori works?
Define first the size of itemset. Do you want to see patterns for a 2-itemset, etc.?
The second is support or the number of transactions containing the itemset divided by the total number of transactions. An itemset that meets the support is called a frequent itemset.

The third is your confidence or the conditional probability of some item, given you have certain other items in your itemset.
A good example is, given chips is in your itemset, there is a 67% confidence of having soda also in the itemset.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

In this problem, you will come up with a decision tree to predict if a planet is habitable based only on features observed by the telescope.

In the table below, you are given the data from all 800 planets surveyed so far. The features observed by telescope are Size ("Big" or "Small"), and Orbit ("Near" or "Far"). Each row indicates the values of the features and habitability, and how many times that set of values was observed. So, for example, there were 20 "Big" planets "Near" their star that were habitable.

Size	Orbit	Habitable	Count
Big	Near	Yes	20
Big	Far	Yes	170
Small	Near	Yes	139
Small	Far	Yes	45
Big	Near	No	130
Big	Far	No	30
Small	Near	No	11
Small	Far	No	255

a. Determine if Planet=Big and Orbit=Far is habitable? Derive and draw the decision tree learned by ID3 on this data.

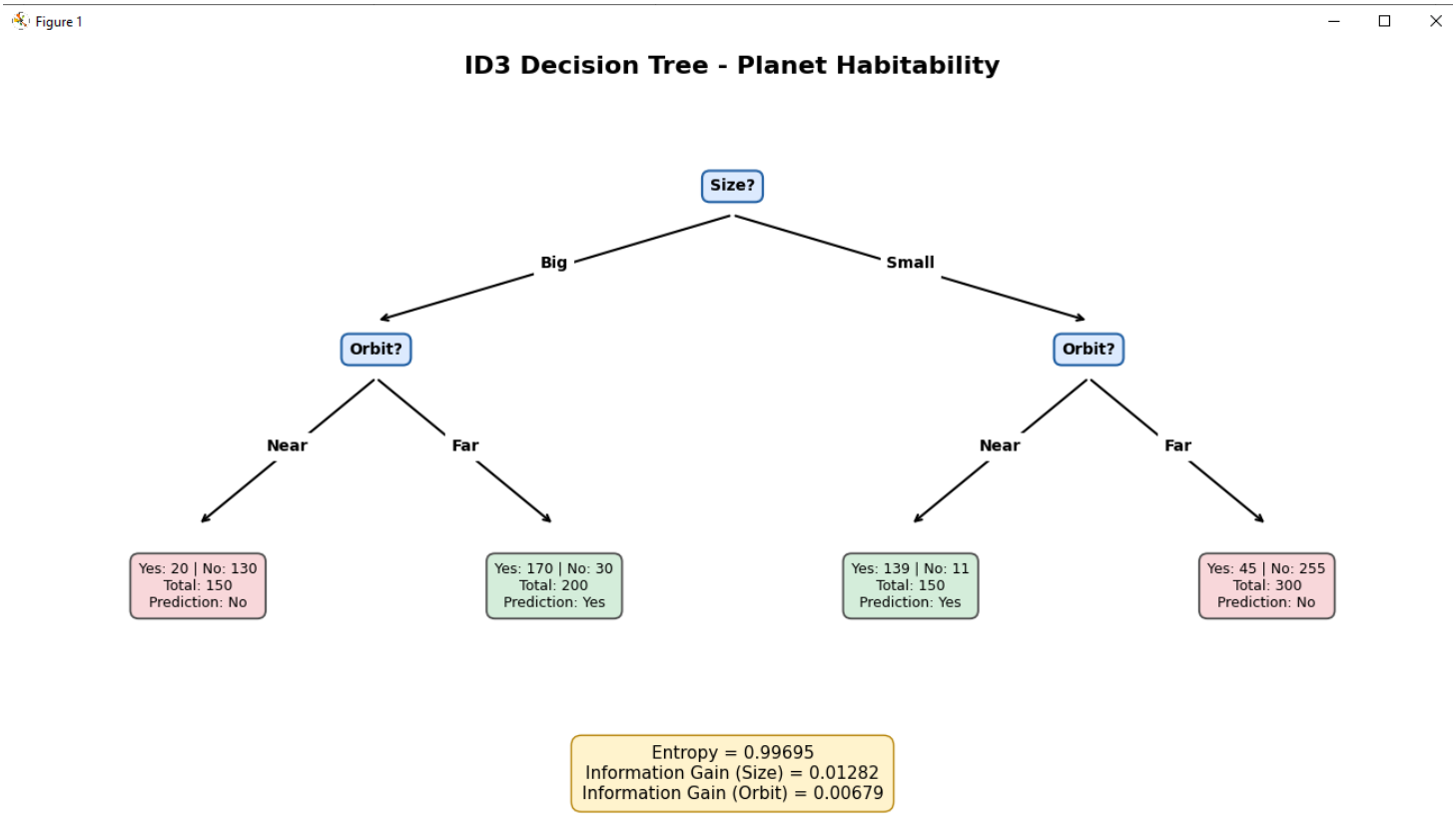
b. Determine if Planet=Small and Orbit=Near is habitable? Derive and draw the decision tree learned by ID3 on this data.

Derive a decision tree learned by ID3 on this data (use the maximum information gain criterion for splits, don't do any pruning). Make sure to clearly mark at each node what attribute you are splitting on, and which value corresponds to which branch. By each leaf node of the tree, write in the number of habitable and inhabitable planets in the training data that belong to that node.

OUTPUT:

```
PLANET HABITABILITY - ID3
-------------------------
Total planets: 800
Habitable: 374
Inhabitable: 426
Dataset Entropy: 0.99695
Information Gain (Size): 0.01282
Information Gain (Orbit): 0.00679

PREDICTIONS
Big, Far: Yes
Small, Near: Yes
```



CODE:

```
# Decision Tree using ID3

import math
import matplotlib.pyplot as plt

# Dataset: Size, Orbit, Habitable, Count
data = [
    ("Big", "Near", "Yes", 20),
    ("Big", "Far", "Yes", 170),
    ("Small", "Near", "Yes", 139),
    ("Small", "Far", "Yes", 45),
    ("Big", "Near", "No", 130),
    ("Big", "Far", "No", 30),
    ("Small", "Near", "No", 11),
    ("Small", "Far", "No", 255)
]

# Calculate entropy
def entropy(yes, no):
    total = yes + no
    result = 0

    for count in [yes, no]:
        if count > 0:
            probability = count / total
            result -= probability * math.log2(probability)

    return result

# Calculate total counts
yes_total = sum(row[3] for row in data if row[2] == "Yes")
no_total = sum(row[3] for row in data if row[2] == "No")
total = yes_total + no_total

base_entropy = entropy(yes_total, no_total)

# Calculate information gain
```

```
    def information_gain(attribute_index):
        groups = {}

        for row in data:
            attribute = row[attribute_index]

            if attribute not in groups:
                groups[attribute] = [0, 0]

            if row[2] == "Yes":
                groups[attribute][0] += row[3]
            else:
                groups[attribute][1] += row[3]

        weighted_entropy = 0

        for yes, no in groups.values():
            count = yes + no
            weighted_entropy += (
                count / total
            ) * entropy(yes, no)

        return base_entropy - weighted_entropy


    # Display calculations
    size_gain = information_gain(0)
    orbit_gain = information_gain(1)

    print("PLANET HABITABILITY - ID3")
    print("-------------------------")
    print("Total planets:", total)
    print("Habitable:", yes_total)
    print("Inhabitable:", no_total)
    print("Dataset Entropy:", round(base_entropy, 5))
    print("Information Gain (Size):", round(size_gain, 5))
    print("Information Gain (Orbit):", round(orbit_gain, 5))

    # Prediction function
```

```
def predict(size, orbit):
    counts = {
        ("Big", "Near"): (20, 130),
        ("Big", "Far"): (170, 30),
        ("Small", "Near"): (139, 11),
        ("Small", "Far"): (45, 255)
    }

    yes, no = counts[(size, orbit)]
    return "Yes" if yes > no else "No"


print("\nPREDICTIONS")
print("Big, Far:", predict("Big", "Far"))
print("Small, Near:", predict("Small", "Near"))


# -------------------------------------
# DRAW THE DECISION TREE USING MATPLOTLIB
# -------------------------------------

fig, ax = plt.subplots(figsize=(13, 8))

ax.set_xlim(0, 12)
ax.set_ylim(0, 10)
ax.axis("off")

# Draw a node with a box and text
def draw_node(x, y, text, color="#DCEAFE",
              width=1.8, height=0.65,
              fontsize=10):
    ax.text(
        x, y, text,
        ha="center", va="center",
        fontsize=fontsize,
        fontweight="bold",
        bbox=dict(
            boxstyle="round,pad=0.5",
            facecolor=color,
```

```
            edgecolor="#2563A6",
            linewidth=1.5
        )
    )

# Draw a branch with its label
def draw_branch(x1, y1, x2, y2, label):
    ax.annotate(
        "",
        xy=(x2, y2 + 0.35),
        xytext=(x1, y1 - 0.35),
        arrowprops=dict(
            arrowstyle="->",
            color="black",
            linewidth=1.5
        )
    )

    mid_x = (x1 + x2) / 2
    mid_y = (y1 + y2) / 2

    ax.text(
        mid_x, mid_y,
        label,
        fontsize=10,
        fontweight="bold",
        ha="center",
        bbox=dict(facecolor="white", edgecolor="none")
    )


# Root node
draw_node(6, 9, "Size?")

# Second-level nodes
draw_node(3, 7, "Orbit?")
draw_node(9, 7, "Orbit?")

# Connect root to second-level nodes
```

```
draw_branch(6, 9, 3, 7, "Big")
draw_branch(6, 9, 9, 7, "Small")

# Leaf positions and their counts
leaves = [
    (1.5, "Near", "Yes: 20 | No: 130\nTotal: 150",
     "Prediction: No", "#F8D7DA"),
    (4.5, "Far", "Yes: 170 | No: 30\nTotal: 200",
     "Prediction: Yes", "#D4EDDA"),
    (7.5, "Near", "Yes: 139 | No: 11\nTotal: 150",
     "Prediction: Yes", "#D4EDDA"),
    (10.5, "Far", "Yes: 45 | No: 255\nTotal: 300",
     "Prediction: No", "#F8D7DA")
]

# Connect Orbit nodes to the four leaves
draw_branch(3, 7, 1.5, 4.5, "Near")
draw_branch(3, 7, 4.5, 4.5, "Far")
draw_branch(9, 7, 7.5, 4.5, "Near")
draw_branch(9, 7, 10.5, 4.5, "Far")

# Draw the leaf boxes
for x, orbit, counts, prediction, color in leaves:
    ax.text(
        x, 4.1,
        counts + "\n" + prediction,
        ha="center",
        va="center",
        fontsize=9,
        bbox=dict(
            boxstyle="round,pad=0.6",
            facecolor=color,
            edgecolor="#555555",
            linewidth=1.3
        )
    )

# Title and information gain
ax.set_title(
```

```
        "ID3 Decision Tree - Planet Habitability",
        fontsize=16,
        fontweight="bold",
        pad=20
    )

    ax.text(
        6, 1.8,
        f"Entropy = {base_entropy:.5f}\n"
        f"Information Gain (Size) = {size_gain:.5f}\n"
        f"Information Gain (Orbit) = {orbit_gain:.5f}",
        ha="center",
        va="center",
        fontsize=11,
        bbox=dict(
            boxstyle="round,pad=0.6",
            facecolor="#FFF3CD",
            edgecolor="#B8860B"
        )
    )

    plt.tight_layout()
    plt.show()
```

CODE SCREENSHOT:

TA7_Liggayu.py

D: > Feu Alabangseu SY 26 - 27 4th year > first sem > CS Elective - Intelligent Systems > Lab act > Technical Assess

```python

# Decision Tree using ID3

import math
import matplotlib.pyplot as plt

# Dataset: Size, Orbit, Habitable, Count
data = [
    ("Big", "Near", "Yes", 20),
    ("Big", "Far", "Yes", 170),
    ("Small", "Near", "Yes", 139),
    ("Small", "Far", "Yes", 45),
    ("Big", "Near", "No", 130),
    ("Big", "Far", "No", 30),
    ("Small", "Near", "No", 11),
    ("Small", "Far", "No", 255)
]

# Calculate entropy
def entropy(yes, no):
    total = yes + no
    result = 0

    for count in [yes, no]:
        if count > 0:
            probability = count / total
            result -= probability * math.log2(probability)

    return result

# Calculate total counts
yes_total = sum(row[3] for row in data if row[2] == "Yes")
no_total = sum(row[3] for row in data if row[2] == "No")
total = yes_total + no_total

base_entropy = entropy(yes_total, no_total)

# Calculate information gain
def information_gain(attribute_index):
    groups = {}

    for row in data:
        attribute = row[attribute_index]

        if attribute not in groups:
            groups[attribute] = [0, 0]
```

```
47
48          if row[2] == "Yes":
49              groups[attribute][0] += row[3]
50          else:
51              groups[attribute][1] += row[3]
52
53      weighted_entropy = 0
54
55      for yes, no in groups.values():
56          count = yes + no
57          weighted_entropy += (
58              count / total
59          ) * entropy(yes, no)
60
61      return base_entropy - weighted_entropy
62
63
64  # Display calculations
65  size_gain = information_gain(0)
66  orbit_gain = information_gain(1)
67
68  print("PLANET HABITABILITY - ID3")
69  print("------------------------")
70  print("Total planets:", total)
71  print("Habitable:", yes_total)
72  print("Inhabitable:", no_total)
73  print("Dataset Entropy:", round(base_entropy, 5))
74  print("Information Gain (Size):", round(size_gain, 5))
75  print("Information Gain (Orbit):", round(orbit_gain, 5))
76
77  # Prediction function
78  def predict(size, orbit):
79      counts = {
80          ("Big", "Near"): (20, 130),
81          ("Big", "Far"): (170, 30),
82          ("Small", "Near"): (139, 11),
83          ("Small", "Far"): (45, 255)
84      }
85
86      yes, no = counts[(size, orbit)]
87      return "Yes" if yes > no else "No"
```

```
print("\nPREDICTIONS")
print("Big, Far:", predict("Big", "Far"))
print("Small, Near:", predict("Small", "Near"))


# ------------------------------------
# DRAW THE DECISION TREE USING MATPLOTLIB
# ------------------------------------

fig, ax = plt.subplots(figsize=(13, 8))

ax.set_xlim(0, 12)
ax.set_ylim(0, 10)
ax.axis("off")

# Draw a node with a box and text
def draw_node(x, y, text, color="#DCEAFE",
              width=1.8, height=0.65,
              fontsize=10):
    ax.text(
        x, y, text,
        ha="center", va="center",
        fontsize=fontsize,
        fontweight="bold",
        bbox=dict(
            boxstyle="round,pad=0.5",
            facecolor=color,
            edgecolor="#2563A6",
            linewidth=1.5
        )
    )

# Draw a branch with its label
def draw_branch(x1, y1, x2, y2, label):
    ax.annotate(
        "",
        xy=(x2, y2 + 0.35),
        xytext=(x1, y1 - 0.35),
        arrowprops=dict(
            arrowstyle="->",
            color="black",
            linewidth=1.5
        )
    )

```

```
    mid_x = (x1 + x2) / 2
    mid_y = (y1 + y2) / 2

    ax.text(
        mid_x, mid_y,
        label,
        fontsize=10,
        fontweight="bold",
        ha="center",
        bbox=dict(facecolor="white", edgecolor="none")
    )


# Root node
draw_node(6, 9, "Size?")

# Second-level nodes
draw_node(3, 7, "Orbit?")
draw_node(9, 7, "Orbit?")

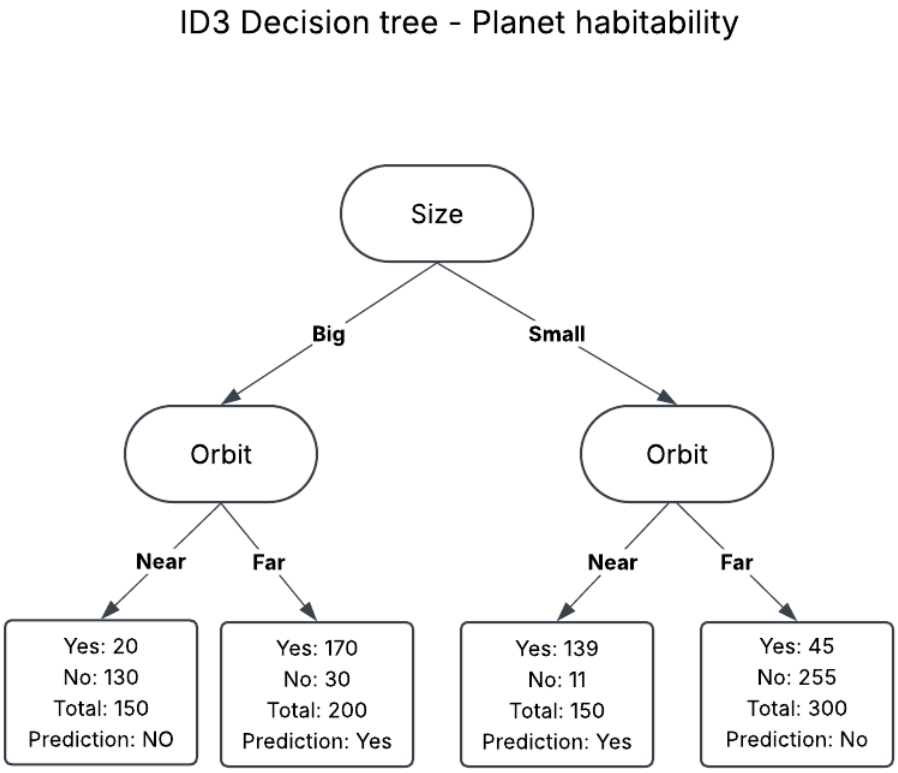
# Connect root to second-level nodes
draw_branch(6, 9, 3, 7, "Big")
draw_branch(6, 9, 9, 7, "Small")

# Leaf positions and their counts
leaves = [
    (1.5, "Near", "Yes: 20 | No: 130\nTotal: 150",
     "Prediction: No", "#F8D7DA"),
    (4.5, "Far", "Yes: 170 | No: 30\nTotal: 200",
     "Prediction: Yes", "#D4EDDA"),
    (7.5, "Near", "Yes: 139 | No: 11\nTotal: 150",
     "Prediction: Yes", "#D4EDDA"),
    (10.5, "Far", "Yes: 45 | No: 255\nTotal: 300",
     "Prediction: No", "#F8D7DA")
]

# Connect Orbit nodes to the four leaves
draw_branch(3, 7, 1.5, 4.5, "Near")
draw_branch(3, 7, 4.5, 4.5, "Far")
draw_branch(9, 7, 7.5, 4.5, "Near")
draw_branch(9, 7, 10.5, 4.5, "Far")
```

```
176
177   # Draw the leaf boxes
178   for x, orbit, counts, prediction, color in leaves:
179       ax.text(
180           x, 4.1,
181           counts + "\n" + prediction,
182           ha="center",
183           va="center",
184           fontsize=9,
185           bbox=dict(
186               boxstyle="round,pad=0.6",
187               facecolor=color,
188               edgecolor="#555555",
189               linewidth=1.3
190           )
191       )
192
193   # Title and information gain
194   ax.set_title(
195       "ID3 Decision Tree - Planet Habitability",
196       fontsize=16,
197       fontweight="bold",
198       pad=20
199   )
200
201   ax.text(
202       6, 1.8,
203       f"Entropy = {base_entropy:.5f}\n"
204       f"Information Gain (Size) = {size_gain:.5f}\n"
205       f"Information Gain (Orbit) = {orbit_gain:.5f}",
206       ha="center",
207       va="center",
208       fontsize=11,
209       bbox=dict(
210           boxstyle="round,pad=0.6",
211           facecolor="#FFF3CD",
212           edgecolor="#B8860B"
213       )
214   )
215
216   plt.tight_layout()
217   plt.show()
218
```

DECISION TREE SCREENSHOT (DRAW):



VII. QUESTION AND ANSWER

Given the details below, apply K-means algorithm to generate iteration 2 and 3. Show your solution.

We will apply k-means on the following 1 dimensional data set for K=2.
Data set {2, 4, 10, 12, 3, 20, 30, 11, 25}

Iteration 1
M1, M2 are the two randomly selected centroids/means where
M1= 4, M2=11
and the initial clusters are
C1= {4}, C2= {11}

Datapoint	D1	D2	Cluster
2	2	9	C1
4	0	7	C1
10	6	1	C2
12	8	1	C2
3	1	8	C1
20	16	9	C2
30	26	19	C2
11	7	0	C2
25	21	14	C2

OUTPUT:

```
Terminal
PS C:\Users\Dom> python -u "d:\Feu Alabangseu SY 2

ITERATION 2
-------------------------------------------
Datapoint   D1        D2        Cluster
2           2.00      9.00      C1
4           0.00      7.00      C1
10          6.00      1.00      C2
12          8.00      1.00      C2
3           1.00      8.00      C1
20          16.00     9.00      C2
30          26.00     19.00     C2
11          7.00      0.00      C2
25          21.00     14.00     C2

C1 = [2, 4, 3]
C2 = [10, 12, 20, 30, 11, 25]
New M1 = 3.0
New M2 = 18.0

ITERATION 3
-------------------------------------------
Datapoint   D1        D2        Cluster
2           1.00      16.00     C1
4           1.00      14.00     C1
10          7.00      8.00      C1
12          9.00      6.00      C2
3           0.00      15.00     C1
20          17.00     2.00      C2
30          27.00     12.00     C2
11          8.00      7.00      C2
25          22.00     7.00      C2

C1 = [2, 4, 10, 3]
C2 = [12, 20, 30, 11, 25]
New M1 = 4.75
New M2 = 19.6
PS C:\Users\Dom>
```

CODE:

```
# CS054 - Intelligent Systems
# K-means Clustering: Iteration 2 and 3

# Dataset
data = [2, 4, 10, 12, 3, 20, 30, 11, 25]

# Initial centroids from Iteration 1
M1 = 4
M2 = 11


# Function to perform one iteration
def kmeans_iteration(data, M1, M2, iteration):

    cluster1 = []
    cluster2 = []

    print(f"\nITERATION {iteration}")
    print("-" * 45)
    print(f"{'Datapoint':<12}{'D1':<10}{'D2':<10}Cluster")

    # Assign data points to the nearest centroid
    for point in data:
        D1 = abs(point - M1)
        D2 = abs(point - M2)

        if D1 <= D2:
            cluster1.append(point)
            cluster = "C1"
        else:
            cluster2.append(point)
            cluster = "C2"

        print(f"{point:<12}{D1:<10.2f}{D2:<10.2f}{cluster}")

    # Calculate new centroids
    new_M1 = sum(cluster1) / len(cluster1)
    new_M2 = sum(cluster2) / len(cluster2)

    print("\nC1 =", cluster1)
    print("C2 =", cluster2)
    print("New M1 =", round(new_M1, 2))
    print("New M2 =", round(new_M2, 2))

    return new_M1, new_M2


# Iteration 2
```

```
M1, M2 = kmeans_iteration(data, M1, M2, 2)

# Iteration 3
M1, M2 = kmeans_iteration(data, M1, M2, 3)
```

CODE SCREENSHOT:

```python

# CS054 - Intelligent Systems
# K-means Clustering: Iteration 2 and 3

# Dataset
data = [2, 4, 10, 12, 3, 20, 30, 11, 25]

# Initial centroids from Iteration 1
M1 = 4
M2 = 11


# Function to perform one iteration
def kmeans_iteration(data, M1, M2, iteration):

    cluster1 = []
    cluster2 = []

    print(f"\nITERATION {iteration}")
    print("-" * 45)
    print(f"{'Datapoint':<12}{'D1':<10}{'D2':<10}Cluster")

    # Assign data points to the nearest centroid
    for point in data:
        D1 = abs(point - M1)
        D2 = abs(point - M2)

        if D1 <= D2:
            cluster1.append(point)
            cluster = "C1"
        else:
            cluster2.append(point)
            cluster = "C2"

        print(f"{point:<12}{D1:<10.2f}{D2:<10.2f}{cluster}")

    # Calculate new centroids
    new_M1 = sum(cluster1) / len(cluster1)
    new_M2 = sum(cluster2) / len(cluster2)

    print("\nC1 =", cluster1)
    print("C2 =", cluster2)
    print("New M1 =", round(new_M1, 2))
    print("New M2 =", round(new_M2, 2))

    return new_M1, new_M2
```

```python
    return new_M1, new_M2


# Iteration 2
M1, M2 = kmeans_iteration(data, M1, M2, 2)

# Iteration 3
M1, M2 = kmeans_iteration(data, M1, M2, 3)

```

VIII. REFERENCES

- Decision Trees: Some Exercises
  Exemplifying the application of the ID3 algorithm in the presence of both categorical and continue attributes, CMU, 2012 fall, Eric Xing, Aarti Singh, HW1, pr. 1.1
- Li, R.. Top 10 Data Mining Algorithms, Hacker Bits. Retrieved from https://hackerbits.com/data/k-means-data-mining-algorithm/
- Shamil, F.R., 3-Mar-2020. Support, Confidence, Minimum support, Frequent itemset, K-itemset, absolute support in data mining. T4 Tutorials. Retrieved from https://t4tutorials.com/support-confidence-minimum-support-frequent-itemset-in-data-mining/
- Schott, M., 23-April-2019. K-Means Clustering Algorithm for Machine Learning. Retrieved from https://medium.com/capital-one-tech/k-means-clustering-algorithm-for-machine-learning-d1d7dc5de882